

Tutorial 3 - t-tests, Confidence Intervals, and the OLS Estimator (with Simulation)

Wenhui Xu

ECO375

Contents

0. Setup	1
1. One-sample t test in R	2
1.1 Warm-up: <code>t.test()</code> on simulated data	2
1.2 DIY t test (to understand what <code>t.test()</code> is doing)	3
What we want our function to do	3
2. Simulation: does the t statistic behave like a t distribution under H0?	5
Goal	5
3. Applied example: audit study (AUDIT)	7
Interpretation of y	7
Test	7
4. OLS in simple regression (CEOSAL1)	9
4.1 Run the regression with <code>lm()</code>	9
4.2 Replicate OLS coefficients using the formulas	10
5. Simulation: sampling variation of OLS estimates (complete version)	11
DGP	11
Optional intuition: draw multiple fitted lines	13
6. Quick checklist (what you should be able to do)	14

0. Setup

This tutorial builds two “muscle groups” that show up everywhere in econometrics:

- 1) **One-sample t tests** (hypotheses, critical values, p-values, confidence intervals)
- 2) **Simple OLS regression** (run with `lm()`, then replicate coefficients using formulas)

Datasets come from the `wooldridge` package: - `audit` (Urban Institute audit study example; variable $y = b - w$) - `ceosal1` (CEO salary vs. ROE example)

```
# Install once if needed:  
# install.packages("wooldridge")
```

```
library(wooldridge)
```

```
data("audit")  
data("ceosal1")
```

1. One-sample t test in R

1.1 Warm-up: `t.test()` on simulated data

```
set.seed(1)  
x <- rnorm(100, mean = 0, sd = 1)  
  
# Two-sided test of H0: mu = 0 at alpha = 0.05  
t.test(x)
```

```
##  
## One Sample t-test  
##  
## data: x  
## t = 1.2123, df = 99, p-value = 0.2283  
## alternative hypothesis: true mean is not equal to 0  
## 95 percent confidence interval:  
## -0.06933487 0.28710961  
## sample estimates:  
## mean of x  
## 0.1088874
```

```
# One-sided test of H0: mu = 0 vs H1: mu < 0  
t.test(x, alternative = "less")
```

```
##  
## One Sample t-test  
##  
## data: x  
## t = 1.2123, df = 99, p-value = 0.8859  
## alternative hypothesis: true mean is less than 0  
## 95 percent confidence interval:  
## -Inf 0.2580236  
## sample estimates:  
## mean of x  
## 0.1088874
```

```
# One-sided test with alpha = 0.10 (=> 90% CI)  
t.test(x, alternative = "less", conf.level = 0.90)
```

```
##
```

```
## One Sample t-test
##
## data: x
## t = 1.2123, df = 99, p-value = 0.8859
## alternative hypothesis: true mean is less than 0
## 90 percent confidence interval:
##      -Inf 0.2247696
## sample estimates:
## mean of x
## 0.1088874
```

```
# Test H0: mu = 4 (just to see the syntax)
t.test(x, mu = 4)
```

```
##
## One Sample t-test
##
## data: x
## t = -43.321, df = 99, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 4
## 95 percent confidence interval:
## -0.06933487 0.28710961
## sample estimates:
## mean of x
## 0.1088874
```

1.2 DIY t test (to understand what t.test() is doing)

What we want our function to do

Inputs:

- x: numeric sample
- mu0: hypothesized mean under H0
- alpha: significance level
- alternative: "two.sided", "less", or "greater"

Outputs (at least):

- t statistic, df, critical value, p-value, confidence interval, and a decision.

```
my_ttest <- function(x, mu0 = 0, alpha = 0.05,
                     alternative = c("two.sided", "less", "greater")) {

  alternative <- match.arg(alternative)
  x <- as.numeric(x)
  n <- length(x)
  if (n < 2) stop("Need at least 2 observations.")

  xbar <- mean(x)
  s <- sd(x)
  se <- s / sqrt(n)
  df <- n - 1

  t_stat <- (xbar - mu0) / se

  if (alternative == "two.sided") {
    crit <- qt(1 - alpha/2, df)
```

```

p_value <- 2 * (1 - pt(abs(t_stat), df))
CI <- c(xbar - crit * se, xbar + crit * se)
decision <- if (abs(t_stat) > crit) "Reject H0" else "Fail to reject H0"

} else if (alternative == "less") {
  crit <- qt(alpha, df) # left-tail critical value
  p_value <- pt(t_stat, df) # left-tail probability
  CI <- c(-Inf, xbar + qt(1 - alpha, df) * se) # one-sided CI (upper bound)
  decision <- if (t_stat < crit) "Reject H0" else "Fail to reject H0"

} else { # alternative == "greater"
  crit <- qt(1 - alpha, df) # right-tail critical value
  p_value <- 1 - pt(t_stat, df) # right-tail probability
  CI <- c(xbar - qt(1 - alpha, df) * se, Inf) # one-sided CI (lower bound)
  decision <- if (t_stat > crit) "Reject H0" else "Fail to reject H0"
}

list(
  n = n,
  df = df,
  xbar = xbar,
  s = s,
  se = se,
  t = t_stat,
  crit_value = crit,
  p_value = p_value,
  CI = CI,
  alternative = alternative,
  alpha = alpha,
  mu0 = mu0,
  decision = decision
)
}

# quick check vs built-in
my_ttest(x, mu0 = 0, alpha = 0.05, alternative = "two.sided")

```

```

## $n
## [1] 100
##
## $df
## [1] 99
##
## $xbar
## [1] 0.1088874
##
## $s
## [1] 0.8981994
##
## $se
## [1] 0.08981994
##
## $t
## [1] 1.212285
##
## $crit_value

```

```
## [1] 1.984217
##
## $p_value
## [1] 0.2282885
##
## $CI
## [1] -0.06933487 0.28710961
##
## $alternative
## [1] "two.sided"
##
## $alpha
## [1] 0.05
##
## $mu0
## [1] 0
##
## $decision
## [1] "Fail to reject H0"
```

```
t.test(x, mu = 0, alternative = "two.sided")
```

```
##
## One Sample t-test
##
## data: x
## t = 1.2123, df = 99, p-value = 0.2283
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## -0.06933487 0.28710961
## sample estimates:
## mean of x
## 0.1088874
```

2. Simulation: does the t statistic behave like a t distribution under H0?

Goal

If the sample really comes from a normal distribution with mean μ_0 , then the one-sample t statistic

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

follows a t_{n-1} distribution.

We'll check this by simulation.

```
set.seed(123)

R <- 5000
n <- 10
mu0 <- 1
sigma <- 2
alpha <- 0.07
```

```

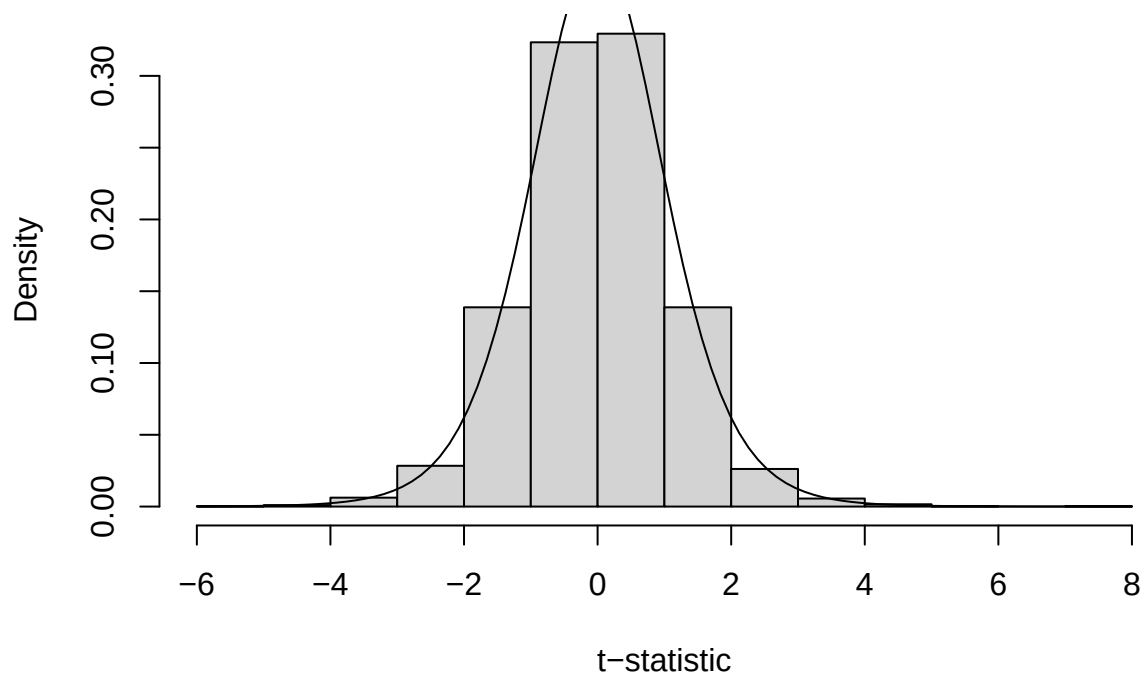
t_stats <- numeric(R)
reject <- logical(R)

for (r in 1:R) {
  x_sim <- rnorm(n, mean = mu0, sd = sigma) # H0 is true
  out <- my_ttest(x_sim, mu0 = mu0, alpha = alpha, alternative = "two.sided")
  t_stats[r] <- out$t
  reject[r] <- (out$decision == "Reject H0")
}

# Compare histogram to theoretical t density
hist(t_stats, probability = TRUE,
     main = "Simulated t-statistic under H0 vs t(df)",
     xlab = "t-statistic")
curve(dt(x, df = n - 1), add = TRUE)

```

Simulated t-statistic under H0 vs t(df)



```

# Empirical Type I error rate (should be close to alpha)
mean(reject)

```

```
## [1] 0.0652
```

```
table(reject)
```

```
## reject
## FALSE TRUE
## 4674 326
```

3. Applied example: audit study (AUDIT)

Interpretation of y

In this dataset, $y = b - w$ is the difference in job-offer indicators between matched applications: - $b = 1$ if the Black applicant gets an offer, 0 otherwise - $w = 1$ if the White applicant gets an offer, 0 otherwise

So if the average difference is negative, it suggests the Black applicant is less likely to receive an offer.

Test

$$H_0 : \mu_y = 0 \quad \text{vs} \quad H_1 : \mu_y < 0.$$

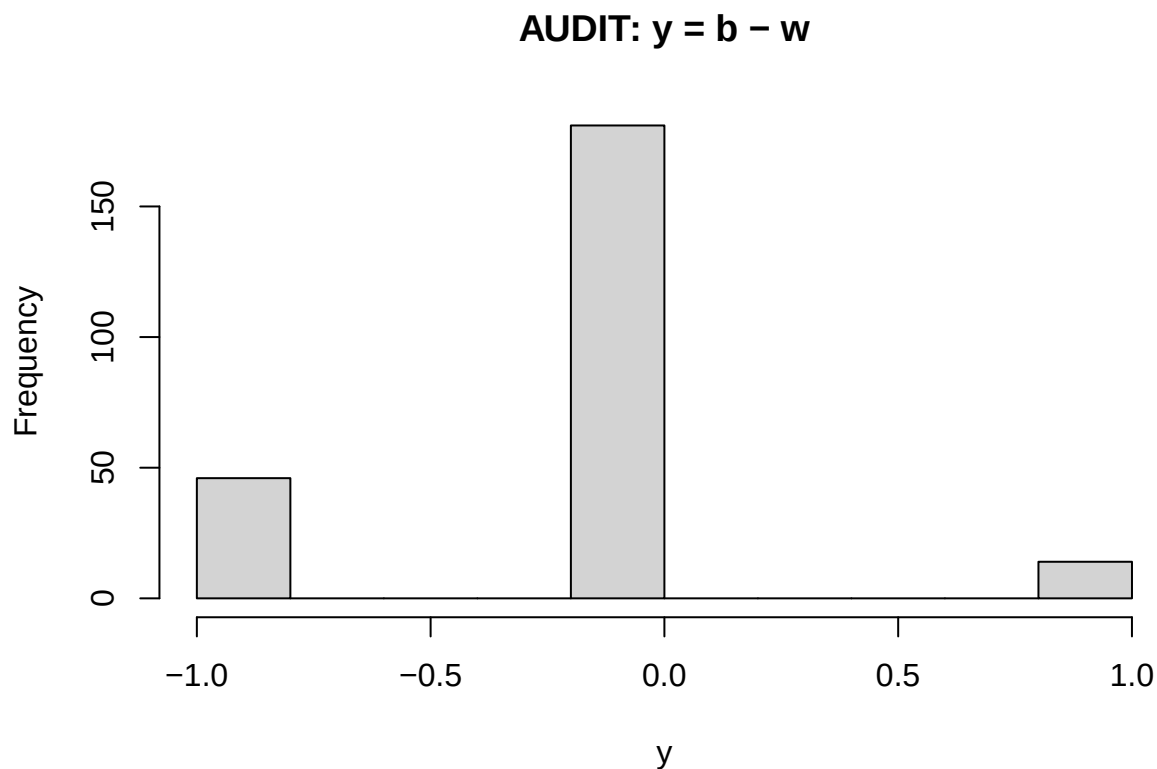
```
summary(audit$y)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -1.0000  0.0000  0.0000 -0.1328  0.0000  1.0000
```

```
table(audit$y)
```

```
##
##  -1    0    1
##  46 181  14
```

```
hist(audit$y, main = "AUDIT: y = b - w", xlab = "y")
```



```
# Built-in
t.test(audit$y, mu = 0, alternative = "less", conf.level = 0.95)
```

```
##
## One Sample t-test
##
## data: audit$y
## t = -4.2768, df = 240, p-value = 1.369e-05
## alternative hypothesis: true mean is less than 0
## 95 percent confidence interval:
## -Inf -0.08151529
## sample estimates:
## mean of x
## -0.1327801
```

```
# DIY
my_ttest(audit$y, mu0 = 0, alpha = 0.05, alternative = "less")
```

```
## $n
## [1] 241
##
## $df
## [1] 240
##
## $xbar
## [1] -0.1327801
##
## $s
## [1] 0.4819709
##
## $se
## [1] 0.03104648
##
## $t
## [1] -4.276816
##
## $crit_value
## [1] -1.651227
##
## $p_value
## [1] 1.369271e-05
##
## $CI
## [1] -Inf -0.08151529
##
## $alternative
## [1] "less"
##
## $alpha
## [1] 0.05
##
## $mu0
## [1] 0
##
## $decision
## [1] "Reject H0"
```


4. OLS in simple regression (CEOSAL1)

4.1 Run the regression with lm()

We estimate:

$$salary_i = \beta_0 + \beta_1 roe_i + u_i.$$

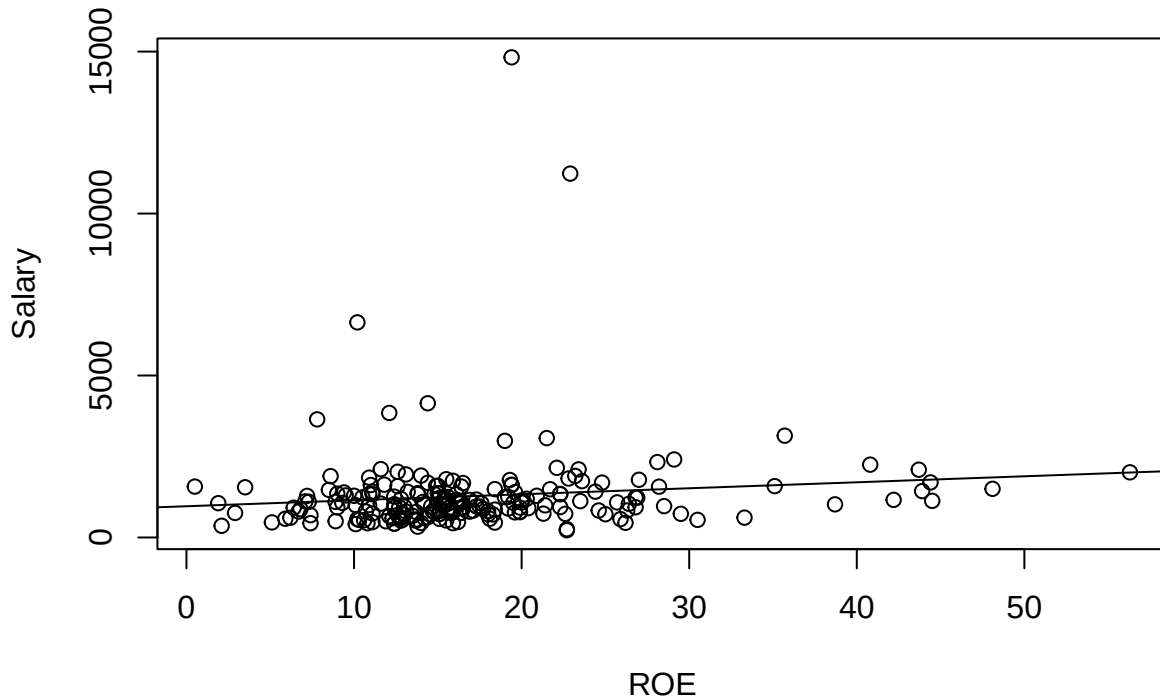
```
plot(salary ~ roe, data = ceosal1,
     main = "CEOSAL1: Salary vs ROE",
     xlab = "ROE", ylab = "Salary")

ols_lm <- lm(salary ~ roe, data = ceosal1)
summary(ols_lm)
```

```
##
## Call:
## lm(formula = salary ~ roe, data = ceosal1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1160.2  -526.0  -254.0   138.8 13499.9
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   963.19     213.24   4.517 1.05e-05 ***
## roe           18.50       11.12   1.663  0.0978 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1367 on 207 degrees of freedom
## Multiple R-squared:  0.01319,    Adjusted R-squared:  0.008421
## F-statistic: 2.767 on 1 and 207 DF,  p-value: 0.09777
```

```
abline(ols_lm)
```

CEOSAL1: Salary vs ROE



```
coef(ols_lm)
```

```
## (Intercept)      roe  
##  963.19134    18.50119
```

4.2 Replicate OLS coefficients using the formulas

The OLS slope and intercept in simple regression are:

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

```
my_ols <- function(x, y) {  
  x <- as.numeric(x)  
  y <- as.numeric(y)  
  if (length(x) != length(y)) stop("x and y must have the same length.")  
  
  xbar <- mean(x)  
  ybar <- mean(y)  
  
  dx <- x - xbar  
  dy <- y - ybar # IMPORTANT: subtract mean(y), not mean(x)  
  
  beta1_hat <- mean(dx * dy) / mean(dx^2)  
  beta0_hat <- ybar - beta1_hat * xbar  
  
  c(beta0_hat = beta0_hat, beta1_hat = beta1_hat)
```

```
}

my_ols(ceosal1$roes, ceosal1$salary)
```

```
## beta0_hat beta1_hat
## 963.19134 18.50119
```

```
coef(ols_lm)
```

```
## (Intercept)      roe
## 963.19134    18.50119
```

5. Simulation: sampling variation of OLS estimates (complete version)

DGP

We simulate data from:

$$y = \beta_0 + \beta_1 x + u, \quad x \sim N(0, 1), \quad u \sim N(0, 1), \quad x \perp u.$$

Then we repeat the sampling process many times and look at the distribution of $\hat{\beta}_0$ and $\hat{\beta}_1$.

```
set.seed(123)

R <- 5000
n <- 100
beta0 <- 1
beta1 <- 2

beta0_hat <- numeric(R)
beta1_hat <- numeric(R)

for (r in 1:R) {
  x_sim <- rnorm(n)
  u_sim <- rnorm(n)
  y_sim <- beta0 + beta1 * x_sim + u_sim

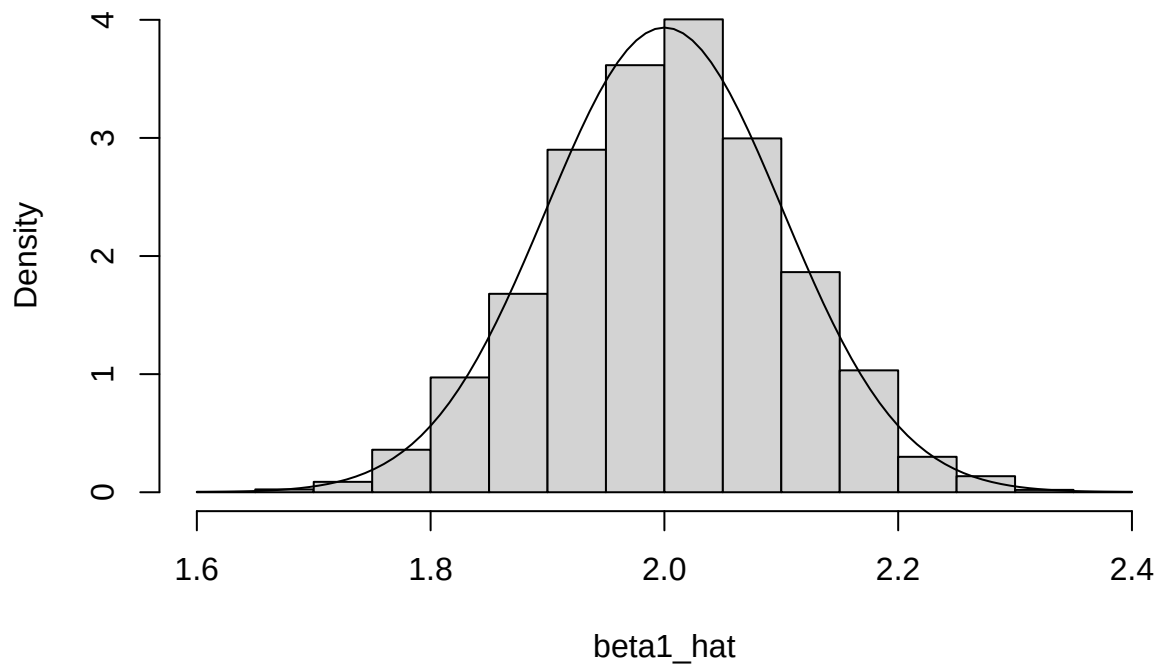
  b <- my_ols(x_sim, y_sim)
  beta0_hat[r] <- b["beta0_hat"]
  beta1_hat[r] <- b["beta1_hat"]
}

c(mean_beta0 = mean(beta0_hat), sd_beta0 = sd(beta0_hat),
  mean_beta1 = mean(beta1_hat), sd_beta1 = sd(beta1_hat))
```

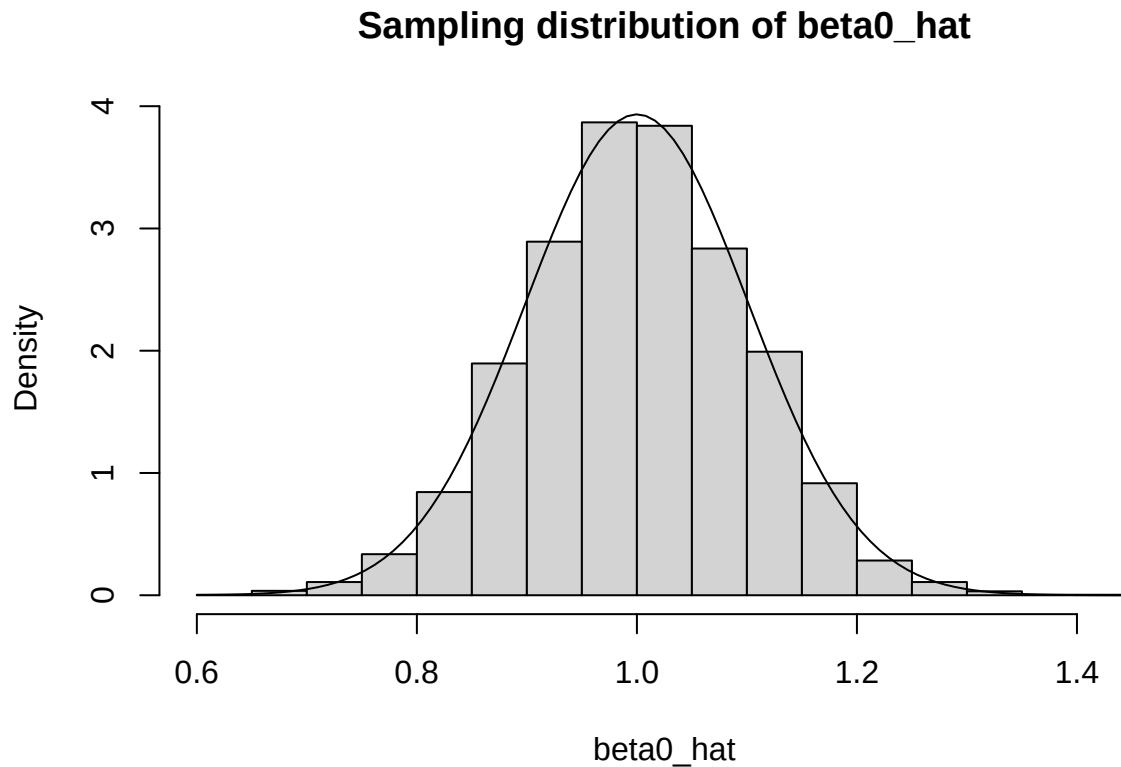
```
## mean_beta0 sd_beta0 mean_beta1 sd_beta1
## 1.0002673 0.1014219 2.0024190 0.1013927
```

```
hist(beta1_hat, probability = TRUE,
     main = "Sampling distribution of beta1_hat",
     xlab = "beta1_hat")
curve(dnorm(x, mean = beta1, sd = sd(beta1_hat)), add = TRUE)
```

Sampling distribution of beta1_hat



```
hist(beta0_hat, probability = TRUE,  
     main = "Sampling distribution of beta0_hat",  
     xlab = "beta0_hat")  
curve(dnorm(x, mean = beta0, sd = sd(beta0_hat)), add = TRUE)
```



Optional intuition: draw multiple fitted lines

```
set.seed(123)

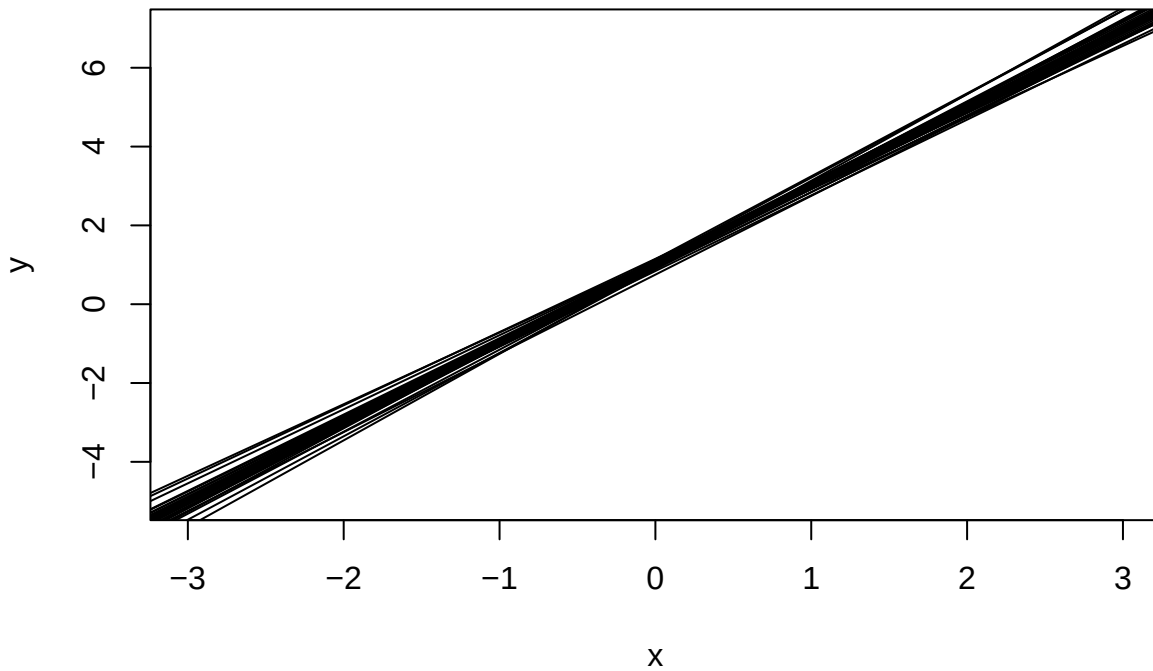
R_lines <- 30
plot(NULL, xlim = c(-3, 3), ylim = c(beta0 - 6, beta0 + 6),
     xlab = "x", ylab = "y",
     main = "Different samples -> different fitted OLS lines")

# True line
abline(a = beta0, b = beta1, lwd = 2)

for (r in 1:R_lines) {
  x_sim <- rnorm(n)
  u_sim <- rnorm(n)
  y_sim <- beta0 + beta1 * x_sim + u_sim

  b <- my_ols(x_sim, y_sim)
  abline(a = b["beta0_hat"], b = b["beta1_hat"])
}
```

Different samples -> different fitted OLS lines



6. Quick checklist (what you should be able to do)

- Use `t.test()` correctly for one-sided and two-sided hypotheses.
- Compute t statistics, critical values (`qt()`), and p-values (`pt()`) manually.
- Explain why the simulated rejection rate under H_0 is close to *alpha*.
- Run `lm()` and interpret the slope and intercept.
- Reproduce OLS coefficients using the simple-regression formulas.
- Use simulation to build intuition for sampling variability.